



Módulo 1

State y Remote Backend

Curso: Terraform Hands-On (GCP)

2 h 35 min (155 min)

Objetivos de Aprendizaje

- Explicar la diferencia entre **state local** y **state remoto**, y los riesgos del primero en equipos.
- Configurar un backend **GCS** con *versioning* y *locking* para un proyecto Terraform.
- Importar un recurso existente (creado a mano) al state sin drift con `terraform import`.
- Crear y usar **workspaces** para aislar entornos (`default` , `dev` , `prod`).
- Leer un `terraform plan` sin sorpresas (símbolos `+` , `~` , `-` , `-/+`).

0. Introducción rápida a Terraform

0.1 Qué es IaC y por qué Terraform

Infraestructura como código: describir la infraestructura en archivos de texto *versionables*, no en clics de consola.

- **Terraform** (HashiCorp) es una de las herramientas más usadas: declarativa, multi-cloud, con lenguaje propio (**HCL**).
- **Caso AppLocker:** la infra de taquillas vive hoy en GCP "a mano y en la cabeza". Terraform la pone en código, *revisable y repetible*.
- **Por qué no basta con scripts de `gcloud`:** idempotencia, *plan* antes de *apply*, grafo de dependencias, estado controlado.

0.2 Vocabulario mínimo

- **Provider:** plugin que habla con un proveedor (Google, AWS, GitHub). Se declara en `terraform { ... }` y se descarga con `terraform init`.
- **Resource:** bloque HCL que describe **un** recurso real. Ej: `resource "google_storage_bucket" "tf_state" { ... }`.
- **Variable:** entrada parametrizable (string, number, bool, list, map...). `variable "x" {}` → USO `var.x`.
- **Output:** valor que Terraform imprime al final del `apply`.
- **Data source:** lectura de información fuera del control de Terraform. `data "..."`.
- **State:** el "mapa" interno de Terraform (sección 1).

0.3 El ciclo básico: init → plan → apply → destroy

Comandos que el alumno va a usar todo el curso:

```
terraform init      # Descarga providers y módulos. Crea .terraform/. Solo una vez.
terraform plan      # Calcula el diff HCL ↔ state. NO toca nada en la nube.
terraform apply     # Ejecuta el plan (con confirmación o -auto-approve).
terraform destroy   # Tira lo declarado. Pensar dos veces antes de ejecutar.
terraform fmt       # Chequeo de estilo.
terraform validate  # Chequeo de sintaxis.
```

Tip: `fmt` y `validate` no descargan providers — son ideales para chequeos rápidos en CI.

 [Ref. oficial CLI Terraform](#)

0.4 Hola Mundo guiado

Demo en vivo. **Provider** `random`, **NO tocar GCP en este bloque.**

```
terraform {
  required_version = ">= 1.5"
  required_providers {
    random = {
      source = "hashicorp/random"
    }
  }
}

resource "random_id" "session" {
  byte_length = 4
}

output "session_id" {
  value = random_id.session.hex
}
```

1. `terraform init` → ver descarga del provider `random`.
2. `terraform plan` → ver el "+ create".
3. `terraform apply` → ver el output `session_id`.
4. Reaplicar → "No changes" (idempotencia).
5. `terraform destroy` → limpieza.

0.5 Cómo se lee un plan

- `+` crear
- `~` modificar in-place
- `-` destruir
- `-/+` destruir y recrear

Ojo: el plan **muestra qué va a pasar**, pero no ejecuta hasta el `apply`. Si ves un `-` en algo que no esperabas → STOP, lee el plan completo antes de confirmar.

Encuadre del módulo

Por qué empezamos por el state

Por qué el state es lo primero

- Terraform sin state controlado es un **riesgo operativo**.
- En MediaMarkt ya existe infraestructura (AppLocker corre en GCP) y este módulo enseña a meterla bajo IaC de forma **segura y reproducible**.

Al final de las 2h 35min el alumno tendrá:

- un bucket remoto versionado con locking habilitado,
- un recurso importado correctamente al state,
- un workspace separado para no pisar el `default`.

Caso AppLocker — punto de partida: el bucket `applocker-tf-state-<sufijo>` ya existe en GCP, creado por el equipo de plataforma antes de la formación. Los alumnos lo importarán al state y a partir de ahí toda la infra vivirá en código.

1. Teoría — State en Terraform

1.1 Qué es el state y por qué existe

- `terraform.tfstate` : archivo JSON que mapea **recursos reales** → **configuración HCL**.
- Terraform lo usa para calcular *diffs*, planificar cambios y mantener metadatos (IDs, atributos, dependencias).
- **Local state por defecto**: vive en el mismo directorio del proyecto. Sirve para pruebas individuales; **rompe en equipo**.

1.2 Riesgos del state local

- Sin control de versiones del `.tfstate` se **pierden cambios o se pisan** entre compañeros.
- Sin locking: dos `terraform apply` simultáneos **corrompen el state**.
- **Secretos en el state** (passwords, tokens) quedan en texto plano en disco.

Conclusión: en equipo, el state local es un riesgo. Necesitamos state remoto desde el día uno.

1.3 State remoto — requisitos

- **Almacenamiento compartido:** todos los miembros leen/escriben el mismo state.
- **Versionado:** recuperar un state anterior si un apply sale mal.
- **Locking:** garantizar que solo un apply modifica el state a la vez.
- **Cifrado en reposo y en tránsito:** el state puede contener datos sensibles.

 [Ref. oficial — Terraform State](#)

2. Lab 1 — Backend GCS con locking

2.1 Prerrequisitos

- Terraform 1.5+ instalado (`terraform -version`).
- `gcloud` autenticado y proyecto GCP activo.
- Permisos: `storage.admin` en el proyecto.

2.2 Crear el bucket manualmente (estado "pre-Terraform")

```
# Variables del alumno
export TF_STATE_BUCKET="applocker-tf-state-${USER}"
export TF_STATE_REGION="us-central1"

# Crear el bucket (esto es lo que luego importaremos)
gcloud storage buckets create gs://${TF_STATE_BUCKET} \
  --location=${TF_STATE_REGION} \
  --uniform-bucket-level-access
```

2.3 Habilitar versioning en el bucket

```
gcloud storage buckets update gs://${TF_STATE_BUCKET} --versioning
```

Crítico: sin versioning no se puede recuperar un estado anterior si un apply sale mal.

2.4 Configurar el backend en HCL

backend.tf :

```
terraform {  
  backend "gcs" {  
    bucket = "applocker-tf-state-<sufijo>"  
    prefix = "terraform/state"  
  }  
}
```

2.5 Inicializar y verificar

```
terraform init
terraform plan  # No debe haber recursos que gestionar todavía
```

Se debe ver el mensaje de **migración de state local a remoto** al ejecutar `init`.

2.6 Validar el locking

- Pedir a dos alumnos que abran dos terminales.
- Uno hace `terraform plan`.
- El otro intenta `terraform apply` sobre el mismo estado.
- El segundo debe quedar **bloqueado**: *"lock acquired by another operation"*.

 [Ref. oficial — Backend GCS](#)

3. Teoría — Importación de recursos

3.1 El problema del drift y la infra "huérfana"

- Recursos creados por consola, otros IaC o clics manuales: Terraform **no los conoce** pero están facturando y en uso.
- `terraform import` añade el recurso al state **SIN tocar** la infraestructura real.

3.2 Limitaciones de `terraform import`

- Importa **un recurso cada vez** (no bloques completos).
- **No genera el bloque HCL** automáticamente: se debe escribir a mano.
- Hay que asegurarse de que el `resource` declarado coincide exactamente con el recurso real (atributos idénticos).

3.3 Flujo recomendado

1. Escribir el bloque `resource` con los atributos correctos (consultar la API/UI).
2. `terraform import <dirección> <id-del-recurso>`.
3. `terraform plan` para confirmar que no hay drift (no debe proponer cambios).



[Ref. oficial — terraform import](#)

4. Lab 2 — Importar el bucket de state

4.1 Declarar el recurso en HCL

main.tf :

```
resource "google_storage_bucket" "tf_state" {  
  name          = "applocker-tf-state-<sufijo>"  
  location      = "us-central1"  
  force_destroy = false  
  
  uniform_bucket_level_access = true  
  versioning {  
    enabled = true  
  }  
  
  labels = {  
    managed_by = "terraform"  
    module     = "bootstrap"  
  }  
}
```

4.2 Importar el recurso

```
terraform import google_storage_bucket.tf_state applocker-tf-state-<sufijo>
```

4.3 Verificar — el plan debe estar limpio

```
terraform plan  
# Expected: "No changes. Your infrastructure matches the configuration."
```

Si el plan está limpio, el recurso está totalmente bajo control de Terraform.

4.4 Reproducir un error común (didáctico)

- Cambiar el `location` en el HCL.
- Volver a hacer `terraform plan`.
- Terraform propondrá **recrear el bucket**.

Lección: ciertos atributos son `ForceNew` y por eso *import + drift* puede ser destructivo.

4.5 Limpieza del lab

```
terraform destroy    # Solo si el formador confirma.  
gsutil ls gs://applocker-tf-state-<sufijo>    # Versioning sigue activo
```

En este curso el bucket se reutiliza en módulos siguientes; NO destruir.

5. Teoría — Workspaces

5.1 Qué son y para qué sirven

- Los workspaces aíslan **states** dentro del mismo backend y configuración.
- Caso típico: `default` (pruebas), `dev`, `staging`, `prod`.
- Cada workspace tiene su propio `.tfstate` (mismo bucket, distinto `prefix`).

5.2 Workspaces vs. directorios separados

- **Workspaces** = una sola configuración HCL, N states. Bueno para entornos con infraestructura similar.
- **Directorios separados** = N configuraciones, N states. Bueno cuando difieren mucho entre sí (regiones, proveedores, recursos).

5.3 Comandos clave

```
terraform workspace list
terraform workspace new dev
terraform workspace select prod
terraform workspace show
```

5.4 Limitaciones importantes

- Las variables **NO** se pueden cambiar entre workspaces (se comparten).
- Para secretos distintos por entorno, usar `*.tfvars` + variable de entorno `TF_WORKSPACE`.

Error común: usar workspaces para aislar credenciales → **no funciona**. Usar variables o secret manager.

 [Ref. oficial — Workspaces](#)

6. Lab 3 — Workspaces para AppLocker

6.1 Crear los workspaces

```
terraform workspace new dev  
terraform workspace new prod  
terraform workspace select dev
```

6.2 Declarar un segundo recurso parametrizado

`main.tf` (añadir):

```
resource "google_storage_bucket" "artifacts" {  
  name          = "applocker-artifacts-${terraform.workspace}-<sufijo>"  
  location      = "us-central1"  
  force_destroy = true  
  
  labels = {  
    environment = terraform.workspace  
    managed_by  = "terraform"  
  }  
}
```

Tip: `terraform.workspace` interpola el nombre del workspace activo en el nombre del bucket, garantizando aislamiento.

6.3 Aplicar en `dev` y verificar

```
terraform workspace select dev
terraform apply -auto-approve
gsutil ls gs://applocker-artifacts-dev-<sufijo>
```

6.4 Cambiar a `prod` y reaplicar

```
terraform workspace select prod
terraform apply -auto-approve
gsutil ls gs://applocker-artifacts-prod-<sufijo>
```

6.5 Verificar aislamiento

- `terraform workspace list` debe mostrar tres estados independientes.
- En GCS, dentro del bucket de state deben aparecer prefijos separados por workspace.

6.6 Limpieza

```
terraform workspace select dev    && terraform destroy  
terraform workspace select prod    && terraform destroy  
terraform workspace select default
```

7. Cierre, Q&A y siguiente módulo

Resumen del módulo

- **State local vs. remoto:** cuándo cada uno; por qué GCS + locking desde el día uno.
- `terraform import`: flujo correcto (HCL primero, import después, plan limpio).
- **Workspaces:** aislamiento de estados, no de configuraciones.

Lo que el alumno debe tener al terminar

- Bucket `applocker-tf-state-<sufijo>` creado y bajo control de Terraform.
- Backend GCS configurado con *versioning* y *locking*.
- Dos workspaces (`dev` , `prod`) operativos con buckets de artefactos.

Errores comunes que vimos

- `terraform import` sin el bloque HCL declarado → *"resource not found in state"*.
- Olvidar `versioning = true` en el bucket de state → no se puede recuperar un estado anterior.
- Usar workspaces para aislar credenciales → **no funciona**; usar variables / secret manager.

Conexión con el Módulo 2

En M2 veremos cómo extraer la definición del bucket de state a un **módulo reusable**, y cómo versionarlo para que el resto de módulos (Cloud SQL, Compute, etc.) lo consuma en vez de duplicar HCL.

Q&A abiertas

- Resolver dudas sobre bloqueo de state y workspaces.
- Confirmar que todos los alumnos tienen `terraform` y `gcloud` operativos antes del M2.

Resumen

Puntos Clave

- El **state** es el mapa interno de Terraform: sin él no hay diffs, ni planificación, ni trabajo en equipo.
- En equipo, state local es un riesgo: usa **GCS + versioning + locking** desde el día uno.
- `terraform import` adopta un recurso existente al state: **HCL primero, import después, plan limpio**.
- **Workspaces** aíslan states (mismo backend), no configuraciones — no sirven para aislar credenciales.
- Lee siempre el plan antes de aplicar: `+` crea, `~` modifica, `-` destruye, `-/+` recrea.

Referencias

Recursos oficiales

- [Terraform State — Documentación oficial](#)
- [Backend GCS — Documentación oficial](#)
- `terraform import` [— Documentación oficial](#)
- [Workspaces — Documentación oficial](#)
- [CLI de Terraform — Referencia](#)
- [GCP Provider — `google\_storage\_bucket`](#)



¡Gracias!

Módulo 1 Completado

Siguiente: Módulo 2 — Módulos reutilizables (state → módulo versionado)